

Gluwa Credal Loan Contract Security Audit

: Credal Loan Contract Security Audit

March 12, 2026

Revision 1.0

ChainLight@Theori

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

Table of Contents

Gluwa Credal Loan Contract Security Audit	1
Table of Contents	2
Executive Summary	4
Audit Overview	5
Scope	5
Code Revision	5
Severity Categories	6
Status Categories	8
Finding Breakdown by Severity	9
Findings	10
Summary	10
#1 CTCLOAN-001 Incompatible ERC-20 transfer handling in LenderVault for non-standard tokens	11
#2 CTCLOAN-002 Missing token address in LenderVault migration signature validation	14
#3 CTCLOAN-003 Missing lender ownership check in privileged loan management functions	17
#4 CTCLOAN-004 Cumulative exemption amount is not properly enforced in LoanContract._processExemption()	19
#5 CTCLOAN-005 Missing borrower nonce in loan creation signature validation	22
#6 CTCLOAN-006 migrateLoanToken() lacks a clear authorization and migration policy	24
#7 CTCLOAN-007 Minor Suggestions	27
Revision History	30

Executive Summary

From January 26 to February 1, 2026, Theori's ChainLight team conducted a 7-day security audit of Gluwa's Credal Loan contracts. The assessment focused on the loan lifecycle and payment processing architecture, including loan creation and closure, lender- and borrower-authorized operations, exemption handling, payment schedule management, signature verification flows, lender vault balance migration, and ERC-20 token transfer handling.

The review covered authorization boundaries for lender-managed actions, signature validation and binding, exemption limit enforcement, migration approval integrity, payment processing constraints, and consistency between typed data hashes and the corresponding signed parameters.

During the audit, several issues were identified, including missing lender ownership checks in privileged loan management functions, incomplete signature binding in lender balance migration due to omission of the token address, improper handling of non-standard ERC-20 token transfers in `LenderVault`, and exemption validation that enforced only a per-call cap rather than a cumulative limit. In addition, several minor issues and improvement points were identified relating to typed data hash definitions, validation logic, error usage, event correctness, and payment schedule validation.

Audit Overview

Scope

Name	Gluwa Credal Loan Contract Security Audit
Target / Version	Git Repository (<code>gluwa/credal-loan</code>): commit <code>132abebcf944bad12508cfcd8dcaad22246aadbc</code>
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	0	N/A
High	0	N/A
Medium	2	- CTCLOAN-003 - CTCLOAN-005
Low	3	- CTCLOAN-001 - CTCLOAN-002 - CTCLOAN-004
Informational	2	- CTCLOAN-006 - CTCLOAN-007
Note	0	N/A

Findings

Summary

#	ID	Title	Severity	Status
1	CTCLOAN-001	Incompatible ERC-20 transfer handling in <code>LenderVault</code> for non-standard tokens	Low	WIP
2	CTCLOAN-002	Missing token address in <code>LenderVault</code> migration signature validation	Low	Patched
3	CTCLOAN-003	Missing lender ownership check in privileged loan management functions	Medium	Patched
4	CTCLOAN-004	Cumulative exemption amount is not properly enforced in <code>LoanContract._processExemption()</code>	Low	Patched
5	CTCLOAN-005	Missing borrower nonce in loan creation signature validation	Medium	Patched
6	CTCLOAN-006	<code>migrateLoanToken()</code> lacks a clear authorization and migration policy	Informational	Acknowledged
7	CTCLOAN-007	Minor Suggestions	Informational	Patched

#1 CTCLOAN-001 Incompatible ERC-20 transfer handling in

LenderVault for non-standard tokens

ID	Summary	Severity
CTCLOAN-001	LenderVault does not safely support ERC-20 tokens that do not return a boolean value from <code>transfer</code> or <code>transferFrom</code> , which may cause valid token transfers to revert unexpectedly.	Low

Description

The `LenderVault` contract determines the success of token transfers by directly validating the return value of ERC-20 `transfer` and `transferFrom` calls. However, some widely used tokens, such as USDT, do not strictly comply with the ERC-20 specification and do not return a boolean value on transfer operations. In such cases, the current implementation may incorrectly treat a successful transfer as a failure and revert.

This issue affects the `deposit`, `depositFor`, `withdraw`, `withdrawFor`, and `recoverTokens` functions, all of which rely on manual validation of token transfer results.

Impact

Low

The contract may fail to interact with non-standard but commonly used ERC-20 tokens, causing unexpected transaction reverts and reducing compatibility with real-world token implementations. As a result, users may be unable to deposit, withdraw, or recover supported assets when such tokens are used.

Recommendation

It is recommended to replace the current manual transfer result validation with OpenZeppelin's `SafeERC20` library, which safely handles both standard and non-standard ERC-20 token implementations. The `deposit`, `depositFor`, `withdraw`, `withdrawFor`, and `recoverTokens` functions should be updated to use `safeTransfer` and `safeTransferFrom` where appropriate.

Remediation

WIP

#2 CTCLOAN-002 Missing token address in LenderVault migration signature validation

ID	Summary	Severity
CTCLOAN-002	The signed data used in <code>LenderVault.migrateLenderBalance()</code> does not include the token address, which allows the owner to use a lender's signature for one migration context to migrate the balance of a different token.	Low

Description

`LenderVault.migrateLenderBalance()` allows the contract owner to migrate a paused lender's balance for a specific token to `newLender`, provided that a valid signature from the lender is supplied. However, the struct hash used for signature verification is computed as:

```
keccak256(abi.encode(MIGRATION_TYPEHASH, lender, newLender, nonce, deadline))
```

The `token` parameter is not included in the signed data, even though it determines which balance is actually migrated. As a result, the lender's signature is not bound to a specific token.

Because of this, the owner can submit a valid signature from the lender and execute the migration for a token different from the one the lender may have intended to authorize. While this issue is limited by the fact that `migrateLenderBalance()` is restricted to `onlyOwner`, the signed message still does not fully represent the action being authorized.

Impact

Low

Since the function is restricted to the contract owner, this issue does not allow arbitrary third parties to exploit the migration flow. However, it weakens the integrity of the authorization model by allowing the owner to use a valid lender signature for a different token than the one intended by the signer. This may result in unauthorized balance migration under an incorrect asset context.

Recommendation

It is recommended to include the `token` address in the struct hash used for signature verification in `migrateLenderBalance()` so that the lender's authorization is explicitly bound to the specific token being migrated. `MIGRATION_TYPEHASH` should also be updated to reflect the modified signed data structure.

Remediation

Patched

commit: `20f871beea7f1be8e1192413a792b649d1a02b0`

#3 CTCLOAN-003 Missing lender ownership check in privileged loan management functions

ID	Summary	Severity
CTCLOAN-003	Any account with <code>LENDER_ROLE</code> can manage loans associated with other lenders, because several privileged loan management functions do not verify whether the caller is the actual lender of the target loan.	Medium

Description

The contract defines an `onlyLoanLender(bytes32 loanHash)` modifier intended to ensure that only the lender associated with a given loan can perform lender-specific actions on that loan. However, several privileged functions, including `receivePayment`, `exemptLoan`, `exemptLoanAmount`, `updatePaymentSchedule`, and `closeLoan`, are protected only by `onlyRole(LENDER_ROLE)` and do not apply `onlyLoanLender(loanHash)`.

As a result, any account granted `LENDER_ROLE` can arbitrarily operate on loans created or managed by other lenders, regardless of the actual lender recorded for the loan. For example, in `receivePayment`, a caller with `LENDER_ROLE` can update the repayment state of any valid loan by reducing its `remainingBalance` and increasing its `repaidAmount`, even if the caller is unrelated to that loan.

This breaks the expected authorization boundary between different lenders and allows one lender-role holder to interfere with another lender's loan lifecycle and accounting.

Impact

Medium

A lender-role holder can manipulate the state of loans that belong to other lenders, including recording repayments, exempting balances, modifying payment schedules, or closing loans. This may result in unauthorized loan state changes, incorrect repayment records, and loss of integrity in lender-specific loan management.

Recommendation

It is recommended to add the `onlyLoanLender(loanHash)` modifier to all functions that are intended to be executed only by the lender associated with the target loan, including `receivePayment`, `exemptLoan`, `exemptLoanAmount`, `updatePaymentSchedule`, and `closeLoan`. This ensures that having `LENDER_ROLE` alone is not sufficient to manage unrelated loans.

Remediation

Patched

commit: `20f871beaa7f1be8e1192413a792b649d1a02b0`

#4 CTCLOAN-004 Cumulative exemption amount is not properly enforced in `LoanContract._processExemption()`

ID	Summary	Severity
CTCLOAN-004	<code>LoanContract._processExemption()</code> validates the exemption amount only on a per-call basis, which means the intended maximum exemption amount is not strictly enforced across multiple calls.	Low

Description

`LoanContract._processExemption()` validates the exemption amount using:

```
require(exemptAmount <= maxExemptAmount, InvalidAmount());
```

This check only ensures that the exemption amount for the current call does not exceed `maxExemptAmount`. However, it does not account for the amount already exempted previously through earlier calls. Since the contract accumulates exempted amounts through `loan.exemptAmount += exemptAmount`, repeated invocations may cause the total exempted amount to exceed the intended maximum limit associated with the exemption.

To strictly enforce the maximum exemption cap across the full lifetime of the loan, the validation should take the cumulative exempted amount into account.

Impact

Low

This issue weakens the correctness of exemption limit enforcement, as the configured maximum may be exceeded through multiple calls. However, the practical impact is limited because the function is intended to be callable only by the lender associated with the loan, and applying excessive exemptions is economically unfavorable to that lender rather than beneficial. As a result, the issue is unlikely to be exploited in practice, but it may still lead to unintended accounting behavior or inconsistency with the intended exemption policy.

Recommendation

It is recommended to validate the cumulative exempted amount rather than only the per-call amount. Specifically, the current check should be updated to:

```
require(loan.exemptAmount + exemptAmount <= maxExemptAmount, InvalidAmount  
());
```

This ensures that the total exempted amount cannot exceed the intended maximum authorized by the exemption configuration.

Remediation

Patched

commit: 20f871beea7f1be8e1192413a792b649d1a02b0

#5 CTCLOAN-005 Missing borrower nonce in loan creation signature validation

ID	Summary	Severity
CTCLOAN-005	The borrower signature used during loan creation was not bound to <code>borrowerNonces[borrower]</code> , which could allow the same signature to be replayed and reused for unintended loan creation.	Medium

Description

In `LoanContract._prepareAndCreateLoan()`, loan creation relies on `_verifyBorrowerSignature()` to validate the borrower's consent to the loan terms. However, if the signature verification logic does not incorporate `borrowerNonces[borrower]` into the signed message, the borrower's signature is not uniquely bound to a single authorization event.

In that case, a previously valid borrower signature may be replayed to create additional loans using the same signed approval, without requiring fresh consent from the borrower. This weakens the integrity of the signature-based authorization flow and may allow multiple loan creations beyond the borrower's original intent.

To prevent replay, the borrower-specific nonce should be passed into `_verifyBorrowerSignature()` and included in the hashed message used for signature recovery.

Impact

Medium

A valid borrower signature could be reused to create multiple loans without the borrower's explicit intent for each one. This may result in unauthorized loan creation and breaks the expected one-time authorization property of the borrower's signature.

Recommendation

It is recommended to pass `borrowerNonces[borrower]` into `_verifyBorrowerSignature()` and include it in the struct hash calculation used for signature verification. After successful

verification, the nonce should be incremented so that each borrower signature can only be used once.

Remediation

Patched

commit: 20f871beaa7f1be8e1192413a792b649d1a02b0

#6 CTCLOAN-006 `migrateLoanToken()` lacks a clear authorization and migration policy

ID	Summary	Severity
CTCLOAN-006	<code>migrateLoanToken()</code> allows an operator to change a loan's <code>paymentToken</code> without clear approval requirements from either the borrower or lender, and the function does not define how amount adjustments should be handled when the new token has different decimals or economic value.	Informational

Description

The `migrateLoanToken()` function allows an account with `OPERATOR_ROLE` to replace the `paymentToken` of a loan while the loan is in the `Pending` or `Active` state. However, it is unclear why this action can be performed without explicit approval from either the borrower or the lender, even though changing the payment token may materially affect the repayment terms of the loan.

In addition, the function updates only the token address and does not define how the loan's economic terms should be handled when the new token differs from the original one in decimals, denomination, or market value. Without a clearly defined conversion or migration policy, it is unclear whether the resulting loan state remains consistent with the original agreement between the parties.

Since this function appears to be incomplete and still under design, the issue is primarily related to specification clarity and incomplete definition of expected behavior rather than an immediately exploitable implementation flaw.

Impact

Informational

Recommendation

It is recommended to define a clear feature specification for `migrateLoanToken()`, including who must authorize the token migration, under what conditions it may occur, and how loan

amounts should be adjusted when the new token differs in decimals or value. If the feature is retained, the implementation should be aligned with that specification before production use.

Remediation

Acknowledged

The team stated that this function is still in a draft state while the loan operation design is being finalized. They agreed with the concern, but decided to leave the function unchanged for now because the logic may be removed or significantly modified later.

#7 CTCLOAN-007 Minor Suggestions

ID	Summary	Severity
CTCLOAN-007	The description includes multiple suggestions for preventing incorrect settings caused by operational mistakes, mitigating potential issues, and improving code maturity and readability.	Informational

Description

1. `LoanContract.LOAN_TYPEHASH` should be updated to match the function parameters. For example: `Loan(address lender, address borrower, bytes32 termHash, uint256 principalAmount, uint256 payableAmount, address principalToken, address paymentToken, uint256 sequenceId)`.
2. `LoanContract.registerLoanTerm()` does not handle the case where both `isAdd` and `registeredContracts[termContract]` are `true`. If this behavior is not intended, it is recommended to add `require(!(isAdd && registeredContracts[termContract]));`.
3. In `LoanContract.addLender()`, the error thrown when the lender already has `LENDER_ROLE` is `Unauthorized()`. Since `Unauthorized()` is typically used when the caller lacks permission to invoke the function, it is not appropriate for the case where the lender already has the role. It is recommended to replace it with a more suitable error. Similarly, `removeLender()` should also use a more appropriate error instead of `Unauthorized()`.
4. In `addLender()`, the `vault` argument of the `LenderAdded()` event is set to the lender address. It is recommended to either pass the actual vault address or remove the `vault` parameter from the event if it is unnecessary.
5. In `LoanPaymentContract.LOAN_PAYMENT_TYPEHASH`, the final `uint256 deadline` string must be placed immediately after the preceding comma, without any whitespace. It is recommended to update the type string accordingly.
6. `LoanContract.closeLoan()` does not validate the remaining token balance. As a result, a loan can be moved to the `Closed` state even when there is still an outstanding balance, and no repayment-related event is emitted in that case. If this behavior is not intended, it is recommended to add a check such as `if (loan.metadata.status == LoanStatus.Active) require(loan.remainingBalance == 0);`.

7. In `LoanContract.updatePaymentSchedule()`, the check `require(totalScheduled <= loan.payableAmount, InvalidAmount())`; only ensures that `totalScheduled` does not exceed `payableAmount`. However, if the borrower has already made a partial repayment, the remaining amount due will be less than `payableAmount`. It is recommended to validate against `loan.remainingBalance` instead of `loan.payableAmount`.
8. In `LoanPaymentContract.processPayment()`, including `params.remainingBalance` in the signed data introduces an unnecessarily strict constraint. If the borrower makes a token repayment after receiving the lender's signature, the signature may become invalid. It is recommended to remove `remainingBalance` from the signed data.

Impact

Informational

Recommendation

Consider applying the suggestions in the description above.

Remediation

Patched

All issues have been patched as recommended.

commit: `20f871beaa7f1be8e1192413a792b649d1a02b0`

Revision History

Version	Date	Description
1.0	March 12, 2026	Initial version

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

